

Guía Práctica: optimización de consultas en PostgreSQL

Parte I: Análisis del Plan de Ejecución

El gerente de ventas le pidió a un desarrollador junior el reporte clásico de la empresa: "Quiero saber el total facturado por cada cliente de Buenos Aires durante el año 2025".

El desarrollador junior se puso a trabajar en el reporte y armó la siguiente consulta.

Ejecuta la siguiente consulta utilizando el comando EXPLAIN ANALYZE por delante. Esto no solo planificará la consulta, sino que la ejecutará midiendo los tiempos reales:

```
EXPLAIN ANALYZE
SELECT c.nombre, SUM(p.importe) AS total_facturado
FROM clientes c
JOIN pedidos p ON c.id_cliente = p.id_cliente
JOIN provincias pr ON c.id_provincia = pr.id_provincia
WHERE pr.nombre = 'Buenos Aires'
      AND EXTRACT(YEAR FROM p.fecha) = 2025
GROUP BY c.nombre;
```

Responde observando el resultado (texto) del Explain:

- A. ¿Cuánto tiempo total tardó en ejecutarse la consulta (Execution Time)?
- B. Busca las palabras Seq Scan on pedidos y Seq Scan on clientes. ¿Qué significa "Seq Scan" y por qué crees que el motor decidió leer las tablas completas?
- C. ¿Qué algoritmo de JOIN eligió el optimizador (ej. Hash Join, Nested Loop, Merge Join)?

Parte II: Optimización Lógica

El desarrollador Junior escribió la cláusula WHERE utilizando una función de extracción de fechas, lo cual es muy común pero ineficiente, vos revisaste su consulta y le pediste que aplique una modificación:

- Consulta A (Junior): WHERE EXTRACT(YEAR FROM p.fecha) = 2025
 - Consulta B (Senior): WHERE p.fecha >= '2025-01-01' AND p.fecha < '2026-01-01'
1. Ejecuta primero este comando para crear un índice en la fecha: CREATE INDEX idx_pedidos_fecha ON pedidos(fecha);

2. Ejecuta un EXPLAIN ANALYZE usando el filtro de la Consulta A (Junior) y anota el tiempo y si usó el índice (Index Scan) o leyó toda la tabla (Seq Scan).
3. Ejecuta un EXPLAIN ANALYZE usando el filtro de la Consulta B (Senior) y anota el tiempo y el método de acceso.

Responde: ¿Por qué la función EXTRACT o YEAR() "ciega" al optimizador y anula el uso del índice?

Parte III: Optimización Física

Volvamos a la consulta original del gerente (Parte I). Actualmente tarda varios segundos porque está escaneando secuencialmente toda la tabla de clientes buscando a los que pertenecen a la provincia de 'Buenos Aires'.

Como vimos en la teoría, indexar las Claves Foráneas (FK) es una regla de oro:

```
CREATE INDEX idx_clientes_provincia ON clientes(id_provincia);
```

Vuelve a ejecutar el EXPLAIN ANALYZE de la consulta original de la Parte I.

Responde:

- a) Observa el plan: ¿Cambió el Seq Scan on clientes por un Bitmap Heap Scan o Index Scan?
- b) ¿Se redujo el tiempo total de ejecución?
- c) ¿Por qué no es una buena idea indexar TODAS las columnas de la tabla "por las dudas"? (Relaciona tu respuesta con el impacto en las operaciones INSERT/UPDATE).

Parte IV: Vistas Estándar vs. Vistas Materializadas

El gerente está muy contento con tu optimización de la consulta, pero dice que el Dashboard hace la petición 100 veces por minuto. Aunque la optimizaste con índices, la CPU del servidor está llegando al 100% por calcular los SUM tantas veces.

1. Crea una vista estándar:

```
CREATE VIEW vista_ventas_bsas_2025 AS
SELECT c.nombre, SUM(p.importe) AS total_facturado
FROM clientes c
JOIN pedidos p ON c.id_cliente = p.id_cliente
JOIN provincias pr ON c.id_provincia = pr.id_provincia
WHERE pr.nombre = 'Buenos Aires' AND p.fecha >= '2025-01-01' AND
p.fecha <= '2025-12-31'
GROUP BY c.nombre;
```

2. Ejecuta EXPLAIN ANALYZE SELECT * FROM vista_ventas_bsas_2025;. ¿Mejóro el tiempo respecto a la consulta cruda? (La respuesta debería ser no. Explica por qué).

3. Creá una Vista Materializada, transforma la solución anterior en una tabla física.

```
CREATE MATERIALIZED VIEW mat_ventas_bsas_2025 AS
SELECT c.nombre, SUM(p.importe) AS total_facturado
FROM clientes c
JOIN pedidos p ON c.id_cliente = p.id_cliente
JOIN provincias pr ON c.id_provincia = pr.id_provincia
WHERE pr.nombre = 'Buenos Aires' AND p.fecha >= '2025-01-01' AND
p.fecha <= '2025-12-31'
GROUP BY c.nombre;
```

4. Ejecuta EXPLAIN ANALYZE SELECT * FROM mat_ventas_bsas_2025;. ¿Cuál es el costo y el tiempo de ejecución ahora?

El problema de la Obsolescencia:

Simula una nueva venta insertando este pedido de un millón de pesos para el 'Cliente 4':

```
UPDATE clientes SET id_provincia=1 WHERE id_cliente=4;

INSERT INTO pedidos (id_cliente, id_articulo, id_sucursal, fecha,
cantidad, importe)
VALUES (4, 1, 1, '2025-06-15', 1, 1000000.00);
```

Vuelve a consultar tu Vista Materializada: SELECT * FROM mat_ventas_bsas_2025 WHERE nombre = 'Cliente 4';.

Responde: ¿Aparece el millón de pesos reflejado en el total? ¿Qué comando SQL exacto debes ejecutar para que la vista se entere de este nuevo dato?